UR Nyarugenge

College of Science and Technology

School of ICT

Computer and Software Engineering

Level3

223007635

# Mobile Application

① Dependencies used and why.

Core Packages

- sqflite . SQLite plugin for flutter
  - . needed for local relational storage without network
  - . provide SQL query API (insert, query, update, delete), transactions, schema.
- Path_Provider . Get persistent storage location on each platform (getApplicationDocumentsDirectory()).
- . Path safe path construction to avoid OS path issues (join()).

Why SQLite is necessary for local data

- Works offline : persists data on the device in a local file
- ACID properties : - Atomic transactions
  - . consistency with constraints
  - . Isolation and durability
- . No server or internet required
- . Lightweight, cross platform
- . Good for structured app data (posts: id | title | content | created AT)

② Database exception handling strategy

Handle database not initialized

- . single DatabaseHelper. instance singleton
- . Database getter : . returns existing_database or
  - runs . initDatabase() to open/create DB with path + config
- . if open fails => Throw descriptive exception and show UI error

Handle insert / update / delete errors.

- . try {...} on DatabaseException catch (e) {throw Exception ('...: ${e.toString()}');}
- . Each CRUD method handles DB errors and rethrows app-level message
- . UI catches and shows snackbar error:
  - . insert / update : save errors
  - . delete : remove failure message.

Handle invalid or corrupted data

- . no del-level validation:
  - . post created with required title / content
- . Form field validators:
  - . non-empty checks

- Map desenialization
  - Post.fromMap uses default fallback in case data missing
  - DateTime.tryparse(...) ?? Datetime.now().
- This reduces crashes on malformed data
  Additional strategy catch generic errors in _loadposts()
  and show read error.

3. SQLite in Flutter

Database vs table

- Database:
  - one file: offline_posts_manager.db
  - Container for multiple tables and database engine metadata
- Table:
  - named set of rows / columns
  - we use posts table:
    - id INTEGER PRIMARY KEY AUTOINCREMENT
    - title TEXT
    - content TEXT
    - createdAT TEXT.

CRUD operations

- Create
  - db.insert(table, post.toMap())
- Read
  - db.query(table, orderBy: 'createdAt DESC')
  - convert rows to models Post.fromMap
- Update
  - db.update(table, post.toMap(), where: 'id = ?', whereArgs: [post.id])
- ~~Delete~~
- Delete
  - db.delete(table, where: 'id = ?', whereArgs: [id])

Flutter asynchronous interaction

- Every DB method is Future<T>
- UI load method: await db.getAllPosts() inside async function.
- setState() runs before / after DB call:
  - show loading spinner → perform async DB I/O → update list
- Async catch error:
  - set error state, show message
- No UI freeze because database methods are non-blocking futures and run on a separate thread via SQLite plugin.

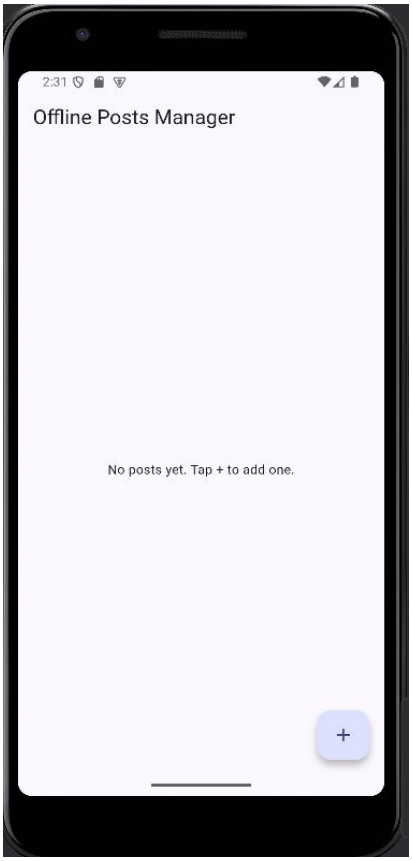
# Screenshots

The screenshots illustrate the workflow of **Offline Posts Manager**, which allows users to create, view, edit, and delete posts without an internet connection.

## 1. Home Screen (Empty State)

The first screen shows the main dashboard with the title *"Offline Posts Manager."*

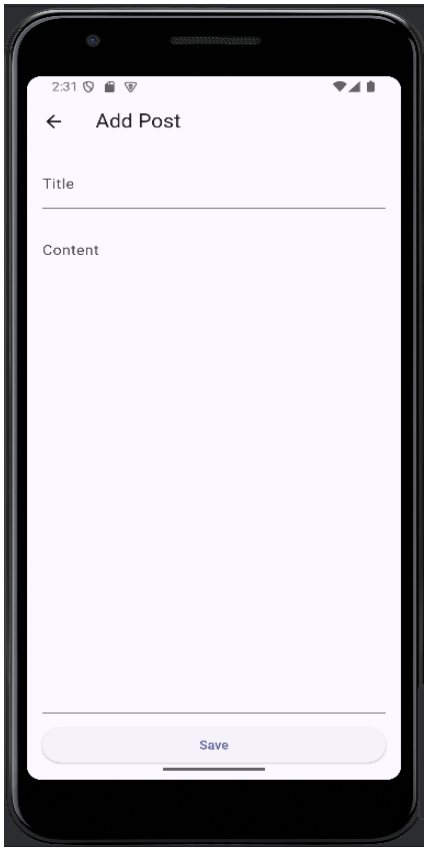
- When there are no posts available, a message appears: *"No posts yet. Tap + to add one."*
- A floating action button (+) is located at the bottom right, enabling users to add a new post.



## 2. Add Post Screen

This screen allows users to create a new post.

- It contains input fields for **Title** and **Content**.
- A **Save** button at the bottom lets users store the post locally.



2. **Home Screen (With Posts)**

After adding a post, the home screen updates to display:

- The **post title** and a preview of the **content**.
- A timestamp indicating when the post was created.
- The floating (+) button remains available for adding more posts.

**4. Post Details Screen**

When a post is selected, a detailed view appears showing:

- Full **title** and **content** of the post.
- Two action buttons:
  - **Edit** – to modify the post
  - **Delete** – to remove the post

**5. Edit Post Screen**

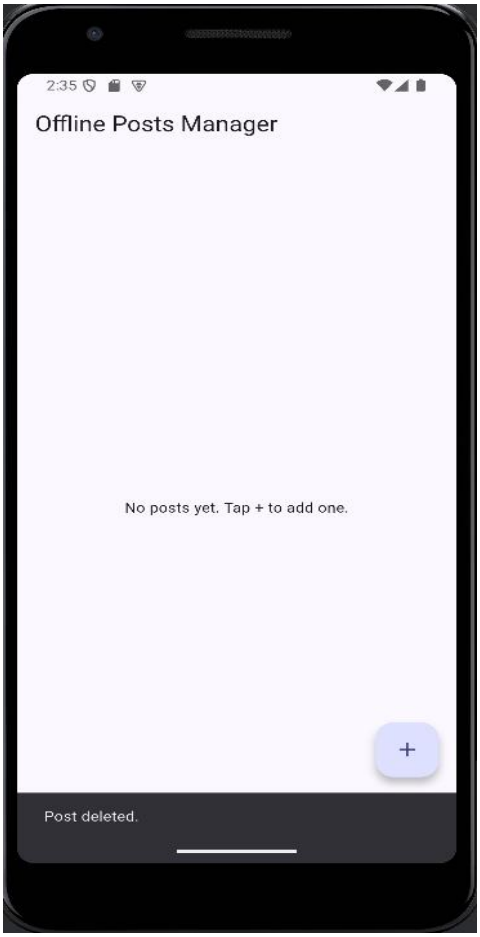
This screen allows users to update an existing post.

- The fields are pre-filled with the current **title** and **content**.
- Users can make changes and tap **Save** to update the post.

**6. Delete Confirmation / Feedback**

After deleting a post:

- The app returns to the home screen.
- A confirmation message (e.g., *"Post deleted"*) appears briefly at the bottom.
- If no posts remain, the empty state message is shown again.




A short video showing app running

https://drive.google.com/file/d/1LB23hPUMsx5B_TJUhy0tIKdlNH3LRD3t/view?usp=sharing